

```
2023-03-17 13:25:25.37224+00
(1 row)

forumdb=> select now();
           now
-----
2023-03-17 13:25:25.37224+00
(1 row)

forumdb=> commit;
COMMIT

forumdb=> begin ;
BEGIN

forumdb=> select now();
           now
-----
2023-03-17 13:27:02.012632+00
(1 row)

forumdb=> commit ;
COMMIT
```

Note: In PostgreSQL 16, when psql shows us a prompt like `*>`, it means that we are inside a transaction block.

2. Now, let's look at an immutable function – for example, the `lower(string_expression)` function. The `lower` function accepts a string and converts it into a lowercase format. As we can see, if the input parameters are the same, the `lower` function always returns the same result, even if it is performed in different transactions:

```
forumdb=> begin;
BEGIN

forumdb=> select now();
```

```
              now
-----
2023-03-17 13:33:39.586388+00
(1 row)

forumdb=> select lower('MICKY MOUSE');
      lower
-----
micky mouse
(1 row)

forumdb=> commit;
COMMIT

forumdb=> begin;
BEGIN

forumdb=> select now();
              now
-----
2023-03-17 13:34:56.491773+00
(1 row)

forumdb=> select lower('MICKY MOUSE');
      lower
-----
micky mouse
(1 row)

forumdb=> commit;
COMMIT
```

## Control structure

PL/pgSQL has the ability to manage control structures such as the following:

- Conditional statements

- Loop statements
- Exception handler statements

## Conditional statements

The PL/pgSQL language can manage IF-type conditional statements and CASE-type conditional statements.

### IF statements

In PL/pgSQL, the syntax of an IF statement is as follows:

```
IF boolean-expression THEN
    statements
[ ELSIF boolean-expression THEN
    statements
[ ELSIF boolean-expression THEN
    statements
    ...
]
[ ELSE
    statements ]
END IF;
```

For example, say we want to write a function that, when given the two input values,  $x$  and  $y$ , returns the following:

- *first parameter is greater than second parameter* if  $x > y$
- *second parameter is greater than first parameter* if  $x < y$
- *the 2 parameters are equals* if  $x = y$

We have to write the following function:

```
forumdb=> CREATE OR REPLACE FUNCTION my_check(x integer default 0, y
integer default 0) RETURNS text AS
$BODY$
BEGIN
    IF x > y THEN
        return 'first parameter is greater than second parameter';
```

```
ELSIF x < y THEN
  return 'second parameter is greater than first parameter';
ELSE
  return 'the 2 parameters are equals';
END IF;
END;
$BODY$
language 'plpgsql';
CREATE FUNCTION
```

In this example, we have seen the IF construct in its largest form: IF [...] THEN[...] ELSIF [...] ELSE[...] ENDIF;

However, shorter forms also exist, as follows:

- IF [...] THEN[...] ELSE[...] ENDIF;
- IF [...] THEN[...] ENDIF;

Some examples of the results provided by the previously defined function are as follows:

```
forumbd=> select my_check(1,2);
              my_check
-----
second parameter is higher than first parameter
(1 row)

forumbd=> select my_check(2,1);
              my_check
-----
first parameter is higher than second parameter
(1 row)

forumbd=> select my_check(1,1);
              my_check
-----
the 2 parameters are equals
(1 row)
```

## CASE statements

In PL/pgSQL, it is also possible to use the CASE statement. The CASE statement can have the following two syntaxes.

The following is a simple CASE statement:

```
CASE search-expression
  WHEN expression [, expression [ ... ]] THEN
    statements
  [ WHEN expression [, expression [ ... ]] THEN
    statements
  ... ]
  [ ELSE
    statements ]
END CASE;
```

The following is a searched CASE statement:

```
CASE
  WHEN boolean-expression THEN
    statements
  [ WHEN boolean-expression THEN
    statements
  ... ]
  [ ELSE
    statements ]
END CASE;
```

Now, we will perform the following operations:

- We will use the first one, the simple CASE syntax, if we have to make a choice from a list of values.
- We will use the second one when we have to choose from a range of values.

Let's start with the first syntax:

```
forumdb=> CREATE OR REPLACE FUNCTION my_check_value(x integer default 0)
RETURNS text AS
$BODY$
BEGIN
```

```
CASE x
WHEN 1 THEN return 'value = 1';
WHEN 2 THEN return 'value = 2';
ELSE return 'value >= 3 ';
END CASE;
END;
$BODY$
language 'plpgsql';
CREATE FUNCTION
```

The preceding `my_check_value` function returns the following:

- `value = 1` if `x = 1`
- `value = 2` if `x = 2`
- `value >= 3` if `x >= 3`

We can see this to be true here:

```
forumdb=> select my_check_value(1);
 my_check_value
-----
value = 1
(1 row)

forumdb=> select my_check_value(2);
 my_check_value
-----
value = 2
(1 row)

forumdb=> select my_check_value(3);
 my_check_value
-----
value >= 3
(1 row)
```

Now, let's see an example of the searched CASE syntax:

```
forumdb=> CREATE OR REPLACE FUNCTION my_check_case(x integer default 0, y
integer default 0) RETURNS text AS
```

```

$BODY$
BEGIN
  CASE
    WHEN x > y THEN return 'first parameter is higher than second
parameter';
    WHEN x < y THEN return 'second parameter is higher than first
parameter';
    ELSE return 'the 2 parameters are equals';
  END CASE;
END;
$BODY$
language 'plpgsql';
CREATE FUNCTION

```

The `my_check_case` function returns the same data as the `my_check` function that we wrote before:

```

forumbd=> select my_check_case(2,1);
              my_check_case
-----
first parameter is higher than second parameter
(1 row)

forumbd=> select my_check_case(1,2);
              my_check_case
-----
second parameter is higher than first parameter
(1 row)

forumbd=> select my_check_case(1,1);
              my_check_case
-----
the 2 parameters are equals
(1 row)

forumbd=> select my_check_case();
              my_check_case
-----

```

```
the 2 parameters are equals
(1 row)
```

## Loop statements

PL/pgSQL can handle loops in many ways. We will look at some examples of how to make a loop next. For further details, we suggest referring to the official documentation at <https://www.postgresql.org/docs/current/plpgsql.html>. What makes PL/pgSQL particularly useful is the fact that it allows us to process data from queries through procedural language. We are going to see now how this is possible.

Suppose that we want to build a PL/pgSQL function that, when given an integer as a parameter, returns a result set of a composite data type. The composite data type that we want it to return is as follows:

|                    |                             |                   |
|--------------------|-----------------------------|-------------------|
| <b>ID</b>          | pk field                    | Integer data type |
| <b>TITLE</b>       | Title field                 | text data type    |
| <b>RECORD_DATA</b> | Title field + content field | hstore data type  |

The right way to build a composite data type is as follows:

```
forumbd=> create type my_ret_type as (
  id integer,
  title text,
  record_data hstore
);
CREATE TYPE
```

The preceding statement creates a new data type, a composite data type, which is composed of an integer data type + a text data type + an hstore data type. Now, if we want to write a function that returns a result set of the `my_ret_type` data type, our first attempt might be as follows:

```
forumbd=> CREATE OR REPLACE FUNCTION my_first_fun (p_id integer) returns
setof my_ret_type as
$$
DECLARE
  rw posts%ROWTYPE; -- declare a rowtype;
  ret my_ret_type;
BEGIN
```

```

    for rw in select * from posts where pk=p_id loop
        ret.id := rw.pk;
        ret.title := rw.title;
        ret.record_data := hstore(ARRAY['title',rw.title,'Title and Content'
                                         ,format('%s %s',rw.title,rw.content)]);
        return next ret;
    end loop;
return;
END;
$$
language 'plpgsql';
CREATE FUNCTION

```

As we can see, many things are concentrated in these few lines of PL/pgSQL code:

1. `rw posts%ROWTYPE`: With this statement, the `rw` variable is defined as a container of a single row of the `posts` table.
2. `for rw in select * from posts where pk=p_id loop`: With this statement, we cycle within the result of the selection, assigning the value returned by the `select` command each time to the `rw` variable. The next three steps assign the values to the `ret` variable.
3. `return next ret;`: This statement returns the value of the `ret` variable and goes to the next record of the `for` cycle.
4. `end loop;`: This statement tells PostgreSQL that the `for` cycle ends here.
5. `return;`: This is the return instruction of the function.



An important thing to remember is that the PL/pgSQL language is inside the PostgreSQL transaction system. This means that the functions are executed atomically and that the function returns the results not at the execution of the `RETURN NEXT` command but at the execution of the `RETURN` command placed at the end of the function. This may mean that for very large datasets, the PL/pgSQL functions can take a long time before returning results.

## The record type

In an example that we used previously, we introduced the `%ROWTYPE` data type. In the PL/pgSQL language, it is possible to generalize this concept. There is a data type called `record` that generalizes the concept of `%ROWTYPE`.

For example, we can rewrite `my_first_fun` in the following way:

```

forumdb=> CREATE OR REPLACE FUNCTION my_second_fun (p_id integer) returns
setof my_ret_type as
$$
DECLARE
    rw record; -- declare a record variable
    ret my_ret_type;
BEGIN
    for rw in select * from posts where pk=p_id loop
        ret.id := rw.pk;
        ret.title := rw.title;
        ret.record_data := hstore(ARRAY['title',rw.title
                                     , 'Title and Content',format('%s %s',rw.title,rw.
content)]);
        return next ret;
    end loop;
    return;
END;
$$
language 'plpgsql';
CREATE FUNCTION

```

The only difference between `my_first_fun` and `my_second_fun` is in this definition:

```
rw record; -- declare a record variable
```

This time, the `rw` variable is defined as a record data type. This means that the `rw` variable is an object that can be associated with any records of any table. The result of the two functions, `my_first_fun` and `my_second_fun`, is the same:

```

forumdb=> \x
Expanded display is on.
forumdb=> select * from my_first_fun(5);
-[ RECORD 1 ]-----
id           | 5
title        | Indexing PostgreSQL
record_data  | "title"=>"Indexing PostgreSQL", "Title and
Content"=>"Indexing PostgreSQL Btree in PostgreSQL is...."

```